

HelixOTA

HelixOTA

ユニバーサルで完全分離型の**OTA**アップデートシステム — 設計段階からのゼロブリック保証

概要

Helix OTAは、ユニバーサルかつ高度に分離されたOTA (Over-The-Air) アップデートシステムです。GoのコントロールプレーンとOSごとのクライアントエージェントで構成され、単一のボードから数百万台規模のデバイス群に至るまで、安全かつ段階的なファームウェア／アプリケーションのアップデートを提供します。最初の対応ターゲットは、Orange Pi 5 Max上のAndroid 15です。

短い説明

Helix OTAは、ユニバーサルなOTAアップデートシステムです。GoのコントロールプレーンとOSごとのクライアントエージェントで構成され、システム破損ゼロ、アップロード検証、きめ細かな段階的ロールアウトを実現します。最初の対応ターゲットはOrange Pi 5 Max上のAndroid 15で、Linux／Windows用アダプターも計画中です。

長い説明

Helix OTAは、ユニバーサルかつ汎用的で、高度に分離されたOTA (Over-The-Air) アップデートシステムであり、唯一絶対の約束を掲げて設計されています。それは、アップデートによって動作中のデバイスが「文鎮化」することは決してあってはならないというものです。このシステムは、Goサーバーのコントロールプレーン、OSごとのクライアント**SDK**／エージェント、そして管理用のダッシュボードで構成され、プラットフォームごとの一から作り直すのではなく、プラグイン可能なOSアダプターを通じてあらゆるオペレーティングシステムに組み込めるよう、ゼロから設計されています。最初の提供ターゲットは、Orange Pi 5 Max上のAndroid 15（全バリエーション）で、ビルドパイプラインはフラッシュ用イメージとともに、検証済みのOTA .zipファイルおよび必須のハッシュ

ファイルを生成します。これにより、デバイスに到達する成果物はすべて検証可能なフィンガープリントを持つことが保証されます。Linux、Windows、その他のOSも、同じアダプターシームの先に控えており、必要なのはアダプターの追加であって、システムの書き換えではありません。

設計は、オペレーターが明示した絶対的な保証を、交渉の余地のないアーキテクチャ上の不変条件として扱うことを基本としています。具体的には、システム破損ゼロ、デプロイ前の成果物の必須検証、きめ細かなロールアウト（全デバイス同時または5/10/30…100%の段階的展開、一時停止・再開制御付き）、フリート全体の可観測性、そして実験室の単一ボードから現場の数百万台規模までの線形スケーラビリティです。この堅牢なアーキテクチャは、デバイス側のネイティブなAndroid A/Bアップデート（AOSPのupdate_engine、AVB/dm-verity、自動ブート失敗時のロールバック機能付き）と、カスタムで分離されたGoコントロールプレーンを組み合わせたものです。これにより、安全性はシリコンに近いブートパスとサーバーの両方に組み込まれ、単一の脆弱なレイヤーに依存することはありません。また、2つのシームが意図的に抽出可能な状態に保たれています。1つはOSアダプターシームで、真のユニバーサルリティを約束するもの、もう1つはロールアウトエンジンシームで、段階的なキャンペーンをOSに依存しない形で実現します。システム全体は6つの公開され、独立してバージョン管理されたota-*サブモジュールに分割されており、モノリシックな構造ではなく、再利用可能なビルディングブロックとして機能します。

Helix OTAは現在、仕様／リサーチおよびテストカバレッジの構築フェーズにあります。リポジトリには、正式な設計文書、ドキュメントエクスポートパイプライン、サブモジュールのスキュアフォールディングが格納されており、その「ブラフ禁止」のガバナンスに則り、完成した本番用サーバーやエージェントはまだ存在しないことが明示されています。現時点で提供されているのは、設計図とそのスキュアフォールディングであり、その旨が正直に明記されています。

コンテンツ

なぜ開発したのか

OTAは通常、デバイスごと、OSごとに再発明され、不具合のあるアップデートがあればデバイス群全体を文鎮化してしまうリスクを抱えています。Helix OTAは、あらゆるOSがアダプターを介して採用可能な、安全性を最優先に設計されたユニバーサルアップデートシステムとして開発されました。ロールバック機能や検証機能は、後付けではなくアーキテクチャに組み込まれた保証として実現されています。

なぜゲームチェンジャーなのか

「デバイスを文鎮化させない」ことや「段階的かつ可観測なロールアウト」を、負荷がかかった際に「うまくいくことを祈る」ベストエフォートの機能として扱うことを拒否しました。これらはブートパスとコントロー

ルプレーンの両方に組み込まれたアーキテクチャ上の不変条件です。さらに、ロールアウトエンジンとOSレイヤーを交換可能なシームとして設計することで、同じコントロールプレーンがAndroidを動かすだけでなく、将来的には他のOSにも対応可能です。アダプターを追加するだけで、フォークもリライトも、信頼済みの安全性保証の再発明も必要ありません。

革新的なポイント

- 二つの抽出可能なシーム - OSアダプターシームとOS非依存のロールアウトエンジンにより、「ユニバーサル」という言葉をマーケティング用語からコードベースの構造的特性へと変貌させました。
- 多層防御の安全性: デバイス側のネイティブA/B (update_engine) + AVB/dm-verity + 自動ブート失敗時のロールバックを、サーバー側の成果物検証の上に重ねて実装。アップデートは複数の独立したゲートを通じてなければ永続化されません。
- カタログ優先、疎結合の設計: 6つの再利用可能で独立してバージョン管理されたota-*サブモジュールに分解。モノリシックなシステムを丸ごと採用するのではなく、必要なものだけを選択して利用できます。
- **HTTP/3 (QUIC)** プライマリトランスポート: 自動HTTP/2フォールバックとBrotli/gzip圧縮のネゴシエーションを備えた、モダンで低遅延な配信システム。障害時には優雅に縮退します。
- プラフ防止エンジニアリング: 設計とステータスは明示的に「仕様策定フェーズ」とマークされ、未実装の機能が「出荷済み」と主張されることはありません。誠実さは、脚注の免責事項ではなく、第一級のエンジニアリング価値として強制されています。

最大の技術的課題とその解決方法

- 不具合のあるアップデートがデバイスを文鎮化させない保証 - OTAにおける最も困難な約束。デバイス側のネイティブAndroid A/Bを必須化することで解決しました。update_engineは非アクティブスロットに書き込みを行いながら、アクティブスロットは稼働を続け、AVB/dm-verityがブートチェーンを暗号的に検証します。新しいスロットのブートに失敗した場合、デバイスは自動的にロールバックします。これらはすべて、サーバー側での成果物検証を必須とすることでバックアップされ、破損したペイロードはサーバーを離れる前に検出されます。
- 一つのシステムで複数のOSに対応 - コアにAndroidの前提を組み込むことを拒否することで解決。プラグ可能なOSアダプターシームがプラットフォーム固有の要素を分離し、OS非依存のロールアウトエンジンシームがキャンペーンロジックの移植性を確保します。それぞれが独立したサブモジュールとして維持されているため、新しいOSへの対応はシステム全体の手術ではなく、追加作業で済みます。
- 段階的で停止可能なロールアウト - 専用のロールアウトエンジンを用いて解決。このエンジンはパーセンテージコホートと成功/エラーの閾値に基づいて動作し、明示的な停止/進行制御を備えています。HTTPとの結合を意図的に排除しているため、同じエンジンがトランスポートに依存せずにキャンペーンを駆動できます。

テックスタック

- **Go + Gin** — 並行処理モデルと軽量なデプロイメントフットプリントを備え、コントロールプレーン、ロールアウトエンジン、アーティファクトバリデータを駆動し、RESTの/api/v1プライマリインターフェースを公開。
- **Kotlin/KMP** — オンデバイスのAndroid OTAエージェントがターゲット間でロジックを共有できるよう選定。デバイスループ全体（ポーリング／ダウンロード／検証／適用／レポート）を管理。
- **HTTP/3 (QUIC) → HTTP/2** — QUICを低遅延かつ回復力のある配信のためのプライマリトランスポートとして採用。モバイル回線のような不安定な環境でも自動的にHTTP/2にフォールバックし、デバイスが取り残されることはない。**Brotli/gzip**はリクエストごとにネゴシエートされ、ペイロードを圧縮。
- **PostgreSQL** — デバイスレジストリ、キャンペーン、テレメトリ間のリレーショナルインテグリティを確保するために選定。フリート状態の正確性が書き込み速度よりも重視される場面で使用。
- **MinIO / S3** — アーティファクトのBLOBストレージとして選定。大容量のファームウェアイメージをコモディティなオブジェクトストレージに保存し、リレーショナルレイヤーから分離。
- **AOSP update_engine + AVB/dm-verity + boot_control** — Androidの実績あるVirtual A/Bおよび検証済みブート機構を再利用することで、独自のアップデータを開発するよりも安全性が高いと判断。デバイス上でのスロットスワップや暗号化ブート検証を実行。
- **React** — オペレータがログインし、アーティファクトをアップロード、ロールアウトを管理、フリートの健全性を一元監視する管理ダッシュボードとして選定。
- **OpenTelemetry + Prometheus/Grafana** — ベンダーニュートラルな計測ツールとして選定。ロールアウトのあらゆる段階をメトリクスやダッシュボードで可視化し、推測に頼らない運用を実現。

ステータスと正直な注意点

- ステータス：開発中。プロジェクトの「ブラフ禁止」ガバナンスに基づき、現時点では動作する本番サーバーやエージェントは存在しない。これは仕様／リサーチおよびテストカバレッジ構築のフェーズである。リポジトリには権威ある設計コーパス、ドキュメントエクスポートパイプライン、サブモジュールのスクャフォールディングが含まれる。
- 6つの公開再利用可能なサブモジュール（ota-protocol、ota-artifact-validator、ota-rollout-engine、ota-update-engine-bridge、ota-android-agent、ota-telemetry-schema）はgithub.com/HelixDevelopment/配下に存在。
- リポジトリ内のテストカバレッジおよびレイテンシの数値は、プロジェクト独自の進行中の記録であり、第三者による確認は行われていない。READMEで引用されているHelixConstitutionの条項番号は未検証。

- ライセンス: **Apache-2.0**。

優先度: **Helix**-プライマリ。